



Universidad Distrital Francisco Jose De Caldas

Facultad de Ingeniería

Ingeniería de Sistemas

Patrones de Diseño de Software

aplicados al contexto de Airbnb

Materia: Modelos de Programación

Estudiante 1: Juan Felipe Guevara Olaya Código: 20231020117

Docente: Baron

30 de mayo de 2026

Bogota, Colombia — 2026

Índice general

Índice general	i
Índice de figuras	xi
Introducción	1
I Patrones Creacionales	3
1. Abstract Factory	4
1.1. Situación Problema	4
1.2. Solución	4
1.3. Roles de las Clases	5
1.4. Main	6
1.5. Salida del Main	6
1.6. Clases	7
1.7. Conclusiones	10
2. Builder	11
2.1. Situación Problema	11
2.2. Solución	11
2.3. Roles de las Clases	12
2.4. Main	12

2.5. Salida del Main	12
2.6. Clases	12
2.6.1. NombreClase	12
2.6.2. NombreClase	12
2.6.3. NombreClase	13
2.7. Conclusiones	13
3. Factory Method	14
3.1. Situación Problema	14
3.2. Solución	14
3.3. Roles de las Clases	15
3.4. Main	15
3.5. Salida del Main	15
3.6. Clases	15
3.6.1. NombreClase	15
3.6.2. NombreClase	15
3.6.3. NombreClase	16
3.7. Conclusiones	16
4. Prototype	17
4.1. Situación Problema	17
4.2. Solución	17
4.3. Roles de las Clases	18
4.4. Main	18
4.5. Salida del Main	18
4.6. Clases	18
4.6.1. NombreClase	18
4.6.2. NombreClase	18

4.6.3. NombreClase	19
4.7. Conclusiones	19
5. Singleton	20
5.1. Situación Problema	20
5.2. Solución	20
5.3. Roles de las Clases	21
5.4. Main	21
5.5. Salida del Main	21
5.6. Clases	21
5.6.1. NombreClase	21
5.6.2. NombreClase	22
5.6.3. NombreClase	22
5.7. Conclusiones	22
II Patrones Estructurales	23
6. Adapter	24
6.1. Situación Problema	24
6.2. Solución	24
6.3. Roles de las Clases	25
6.4. Main	25
6.5. Salida del Main	25
6.6. Clases	25
6.6.1. NombreClase	25
6.6.2. NombreClase	25
6.6.3. NombreClase	25
6.7. Conclusiones	26

7. Bridge	27
7.1. Situación Problema	27
7.2. Solución	27
7.3. Roles de las Clases	28
7.4. Main	28
7.5. Salida del Main	28
7.6. Clases	28
7.6.1. NombreClase	28
7.6.2. NombreClase	29
7.6.3. NombreClase	29
7.7. Conclusiones	29
8. Composite	30
8.1. Situación Problema	30
8.2. Solución	30
8.3. Roles de las Clases	31
8.4. Main	31
8.5. Salida del Main	31
8.6. Clases	31
8.6.1. NombreClase	31
8.6.2. NombreClase	31
8.6.3. NombreClase	32
8.7. Conclusiones	32
9. Decorator	33
9.1. Situación Problema	33
9.2. Solución	33
9.3. Roles de las Clases	34

9.4. Main	34
9.5. Salida del Main	34
9.6. Clases	34
9.6.1. NombreClase	34
9.6.2. NombreClase	34
9.6.3. NombreClase	34
9.7. Conclusiones	35
10.Facade	36
10.1. Situación Problema	36
10.2. Solución	36
10.3. Roles de las Clases	37
10.4. Main	37
10.5. Salida del Main	37
10.6. Clases	37
10.6.1. NombreClase	37
10.6.2. NombreClase	37
10.6.3. NombreClase	37
10.7. Conclusiones	38
11.Flyweight	39
11.1. Situación Problema	39
11.2. Solución	39
11.3. Roles de las Clases	40
11.4. Main	40
11.5. Salida del Main	40
11.6. Clases	40
11.6.1. NombreClase	40

11.6.2. NombreClase	40
11.6.3. NombreClase	40
11.7. Conclusiones	41
12.Proxy	42
12.1. Situación Problema	42
12.2. Solución	42
12.3. Roles de las Clases	43
12.4. Main	43
12.5. Salida del Main	43
12.6. Clases	43
12.6.1. NombreClase	43
12.6.2. NombreClase	43
12.6.3. NombreClase	44
12.7. Conclusiones	44
III Patrones de Comportamiento	45
13.Chain Of Responsibility	46
13.1. Situación Problema	46
13.2. Solución	46
13.3. Diagrama de clases	47
13.4. Roles de las Clases	47
13.5. Main	48
13.6. Salida del Main	49
13.7. Clases	50
13.7.1. NombreClase	50
13.7.2. NombreClase	50

13.7.3. NombreClase	50
13.8. Conclusiones	50
14.Command	51
14.1. Situación Problema	51
14.2. Solución	51
14.3. Roles de las Clases	51
14.4. Main	51
14.5. Salida del Main	51
14.6. Clases	52
14.6.1. NombreClase	52
14.6.2. NombreClase	52
14.6.3. NombreClase	52
14.7. Conclusiones	52
15.Iterator	53
15.1. Situación Problema	53
15.2. Solución	53
15.3. Roles de las Clases	54
15.4. Main	54
15.5. Salida del Main	54
15.6. Clases	54
15.6.1. NombreClase	54
15.6.2. NombreClase	54
15.6.3. NombreClase	54
15.7. Conclusiones	55
16.Mediator	56
16.1. Situación Problema	56

16.2. Solución	56
16.3. Roles de las Clases	57
16.4. Main	57
16.5. Salida del Main	57
16.6. Clases	57
16.6.1. NombreClase	57
16.6.2. NombreClase	57
16.6.3. NombreClase	57
16.7. Conclusiones	58
17.Memento	59
17.1. Situación Problema	59
17.2. Solución	59
17.3. Roles de las Clases	60
17.4. Main	60
17.5. Salida del Main	60
17.6. Clases	60
17.6.1. NombreClase	60
17.6.2. NombreClase	60
17.6.3. NombreClase	60
17.7. Conclusiones	61
18.Observer	62
18.1. Situación Problema	62
18.2. Solución	62
18.3. Roles de las Clases	62
18.4. Main	62
18.5. Salida del Main	63

18.6. Clases	63
18.6.1. NombreClase	63
18.6.2. NombreClase	63
18.6.3. NombreClase	63
18.7. Conclusiones	63
19.State	64
19.1. Situación Problema	64
19.2. Solución	64
19.3. Roles de las Clases	65
19.4. Main	65
19.5. Salida del Main	65
19.6. Clases	65
19.6.1. NombreClase	65
19.6.2. NombreClase	65
19.6.3. NombreClase	65
19.7. Conclusiones	66
20.Strategy	67
20.1. Situación Problema	67
20.2. Solución	67
20.3. Roles de las Clases	68
20.4. Main	68
20.5. Salida del Main	68
20.6. Clases	68
20.6.1. NombreClase	68
20.6.2. NombreClase	68
20.6.3. NombreClase	68

20.7. Conclusiones	69
21.Template Method	70
21.1. Situación Problema	70
21.2. Solución	70
21.3. Roles de las Clases	71
21.4. Main	71
21.5. Salida del Main	71
21.6. Clases	71
21.6.1. NombreClase	71
21.6.2. NombreClase	71
21.6.3. NombreClase	71
21.7. Conclusiones	72
22.Visitor	73
22.1. Situación Problema	73
22.2. Solución	73
22.3. Roles de las Clases	74
22.4. Main	74
22.5. Salida del Main	74
22.6. Clases	74
22.6.1. NombreClase	74
22.6.2. NombreClase	74
22.6.3. NombreClase	75
22.7. Conclusiones	75
Conclusiones Generales	76

Índice de figuras

1.1. Diagrama de clases del patrón Abstract Factory	5
2.1. Diagrama de clases del patrón Abstract Factory	12
3.1. Diagrama de clases del patrón Abstract Factory	15
4.1. Diagrama de clases del patrón Abstract Factory	18
5.1. Diagrama de clases del patrón Abstract Factory	21
6.1. Diagrama de clases del patrón Abstract Factory	25
7.1. Diagrama de clases del patrón Abstract Factory	28
8.1. Diagrama de clases del patrón Abstract Factory	31
9.1. Diagrama de clases del patrón Abstract Factory	34
10.1. Diagrama de clases del patrón Abstract Factory	37
11.1. Diagrama de clases del patrón Abstract Factory	40
12.1. Diagrama de clases del patrón Abstract Factory	43
13.1. Diagrama de clases del patrón Chain of Responsibility	47
14.1. Diagrama de clases del patrón Abstract Factory	51

15.1. Diagrama de clases del patrón Abstract Factory	54
16.1. Diagrama de clases del patrón Abstract Factory	57
17.1. Diagrama de clases del patrón Abstract Factory	60
18.1. Diagrama de clases del patrón Abstract Factory	62
19.1. Diagrama de clases del patrón Abstract Factory	65
20.1. Diagrama de clases del patrón Abstract Factory	68
21.1. Diagrama de clases del patrón Abstract Factory	71
22.1. Diagrama de clases del patrón Abstract Factory	74

Introducción

El desarrollo de software moderno se enfrenta continuamente a desafíos de diseño que, aunque varían en dominio y escala, comparten una naturaleza estructural recurrente. Para responder a estos desafíos de manera sistemática y reutilizable, la comunidad de ingeniería de software adoptó el concepto de **patrones de diseño**, popularizado por Gamma, Helm, Johnson y Vlissides en su obra clásica *Design Patterns: Elements of Reusable Object-Oriented Software* (1994) [?], conocida coloquialmente como «*el libro de la Banda de los Cuatro*» (*GoF*).

Un patrón de diseño es una *solución general y reutilizable* a un problema recurrente dentro de un contexto determinado. No se trata de código listo para copiar y pegar, sino de una plantilla o esquema conceptual que debe adaptarse a cada situación concreta.

Contexto de aplicación: Airbnb

A lo largo de este documento, los 23 patrones del catálogo GoF se analizan y aplican al ecosistema de **Airbnb**, una de las plataformas de alojamiento entre pares (*peer-to-peer*) más grandes del mundo. Airbnb conecta a *anfitriones* que ofrecen espacios habitacionales con *huéspedes* que buscan alojamiento temporal, gestionando además pagos, reseñas, búsquedas geoespaciales, notificaciones y políticas de cancelación, entre otros servicios.

Esta plataforma resulta un escenario idóneo para el estudio de patrones de diseño porque presenta problemáticas reales y complejas:

- **Creación dinámica de objetos:** distintos tipos de alojamiento (apartamento, cabaña, habitación privada, etc.) con atributos y reglas de negocio propios.
- **Integración con servicios externos:** pasarelas de pago, proveedores de mapas, sistemas de correo y notificaciones push.

- **Gestión del estado:** reservas que transitan por estados como *pendiente*, *confirmada*, *activa* y *finalizada*.
- **Escalabilidad y rendimiento:** millones de listados simultáneos que exigen estructuras de datos eficientes y acceso controlado a recursos.
- **Comportamiento flexible:** algoritmos de búsqueda, filtrado y recomendación que varían según preferencias del usuario.

Organización del documento

Los patrones se presentan agrupados en las tres categorías canónicas del GoF:

1. **Patrones Creacionales** (5 patrones): abordan la forma en que se crean los objetos, desacoplando el código cliente de las clases concretas que instancia.
2. **Patrones Estructurales** (7 patrones): describen cómo se componen clases y objetos para formar estructuras más grandes y flexibles.
3. **Patrones de Comportamiento** (10 patrones): se ocupan de los algoritmos y la asignación de responsabilidades entre objetos.

Dentro de cada categoría los patrones se presentan en orden alfabético. Para cada uno se incluye: la situación-problema en el contexto de Airbnb, el diagrama de clases UML, la descripción de roles, un *main* de ejemplo con su salida esperada, el código fuente de las clases y las conclusiones particulares del patrón.

Parte I

Patrones Creacionales

Capítulo 1

Abstract Factory

1.1 Situación Problema

Airbnb opera en tres superficies distintas: aplicación web, aplicación iOS y aplicación Android. Cada superficie tiene sus propios estándares visuales: los botones, tarjetas de propiedad y formularios de reserva difieren en apariencia y comportamiento entre plataformas, pero siempre deben mantenerse visualmente coherentes *dentro* de la misma plataforma.

El equipo de frontend se encontró con que el código de composición de pantallas mezclaba condicionales de plataforma con la lógica de negocio. Crear un componente de tarjeta de propiedad para web y luego reutilizar ese mismo constructor en la app de iOS producía inconsistencias visuales. El problema se agravaba cada vez que se quería incorporar una nueva plataforma: había que rastrear todos los puntos del código donde se instanciaban componentes y añadir una nueva rama condicional.

1.2 Solución

El patrón Abstract Factory define una interfaz **FabricaUI** con métodos para crear cada familia de componentes (`crearBoton()`, `crearTarjetaPropiedad()`, `crearFormularioReserva()`). Las fábricas concretas **FabricaWeb**, **FabricaIOS** y **FabricaAndroid** implementan esa interfaz produciendo componentes coherentes con su plataforma.

El código de composición de pantallas trabaja únicamente contra **FabricaUI**; en tiempo de ejecución se inyecta la fábrica correspondiente a la plataforma activa. Esto garantiza que los componentes de una misma familia siempre sean compatibles entre sí, elimina los condicionales de plataforma dispersos por el código y permite incorporar nuevas superficies

simplemente añadiendo una nueva fábrica concreta.

Figura 1.1. Diagrama de clases del patrón Abstract Factory

1.3 Roles de las Clases

- **AbstractFactory:** La interfaz `FabricaUI` define los métodos necesarios para crear cada tipo de componente de la interfaz gráfica. Su objetivo es garantizar que el cliente pueda trabajar con familias completas de productos sin depender de implementaciones concretas.
- **ConcreteFactory:** Las clases `FabricaWeb`, `FabricaIOS` y `FabricaAndroid` implementan la interfaz `FabricaUI` y se encargan de crear productos compatibles entre sí según la plataforma correspondiente. Cada fábrica representa una familia completa de componentes visuales.
- **AbstractProduct:** Las interfaces `Boton`, `TarjetaPropiedad` y `FormularioReserva` definen el comportamiento común que deben cumplir todos los productos de una misma categoría, independientemente de la plataforma donde se utilicen.
 - **AbstractFactory:** La interfaz `FabricaUI` define los métodos necesarios para crear las diferentes familias de componentes gráficos sin depender de clases concretas.
 - **ConcreteFactory:** Las clases `FabricaWeb`, `FabricaIOS` y `FabricaAndroid` implementan la fábrica abstracta y crean componentes compatibles con cada plataforma específica.
 - **AbstractProduct:** Las interfaces `Boton`, `TarjetaPropiedad` y `FormularioReserva` definen el comportamiento común de los productos de la interfaz.
 - **ConcreteProduct:** Las clases `BotonWeb`, `TarjetaWeb`, `FormularioWeb`, `BotonIOS`, `TarjetaIOS`, `FormularioIOS`, `BotonAndroid`, `TarjetaAndroid` y `FormularioAndroid` representan los productos concretos de cada plataforma.
 - **Client:** La clase `Cliente` utiliza únicamente la interfaz `FabricaUI` para obtener los componentes necesarios. El cliente no conoce las clases concretas que instancia, lo que permite cambiar de plataforma sin modificar la lógica principal de la aplicación.

1.4 Main

AbstractFactory.java

```
1 class AbstractFactory {
2     public static void main(String[] args) {
3         Cliente clienteWEB = new Cliente(new FabricaWeb());
4         Cliente clienteIOS = new Cliente(new FabricaIOS());
5         Cliente clienteAndroid = new Cliente(new FabricaAndroid());
6
7         Cliente[] clientes = {clienteWEB, clienteIOS,
8             ↪ clienteAndroid};
9
10        for (Cliente cliente : clientes) {
11            System.out.println(cliente);
12            System.out.println(cliente.obtenerVista());
13            System.out.println();
14        }
15    }
16}
```

1.5 Salida del Main

```
WEB
Tarjeta Web
Formulario Web
Boton Web

IOS
Tarjeta iOS
Formulario iOS
Boton iOS

ANDROID
Tarjeta Android
Formulario Android
Boton Android
```

1.6 Clases

Abstract Products

```
1 interface Boton {
2     String render();
3 }
4
5 interface TarjetaPropiedad {
6     String render();
7 }
8
9 interface FormularioReserva {
10     String render();
11 }
```

Abstract Factory

```
1 interface FabricaUI {
2     Boton crearBoton();
3     TarjetaPropiedad crearTarjetaPropiedad();
4     FormularioReserva crearFormularioReserva();
5 }
```

Concrete Products

```
1 // Familia Web
2 class BotonWeb implements Boton {
3     public String render() { return "Boton Web"; }
4 }
5 class TarjetaWeb implements TarjetaPropiedad {
6     public String render() { return "Tarjeta Web"; }
7 }
8 class FormularioWeb implements FormularioReserva {
9     public String render() { return "Formulario Web"; }
10 }
```

```
11
12 // Familia iOS
13 class BotonIOS implements Boton {
14     public String render() { return "Boton iOS"; }
15 }
16 class TarjetaIOS implements TarjetaPropiedad {
17     public String render() { return "Tarjeta iOS"; }
18 }
19 class FormularioIOS implements FormularioReserva {
20     public String render() { return "Formulario iOS"; }
21 }
22
23 // Familia Android
24 class BotonAndroid implements Boton {
25     public String render() { return "Boton Android"; }
26 }
27 class TarjetaAndroid implements TarjetaPropiedad {
28     public String render() { return "Tarjeta Android"; }
29 }
30 class FormularioAndroid implements FormularioReserva {
31     public String render() { return "Formulario Android"; }
32 }
```

Concrete Factories

```
1 class FabricaWeb implements FabricaUI {
2     public Boton crearBoton() { return new BotonWeb(); }
3     public TarjetaPropiedad crearTarjetaPropiedad() { return new
    ↪ TarjetaWeb(); }
4     public FormularioReserva crearFormularioReserva() { return new
    ↪ FormularioWeb(); }
5 }
6
7 class FabricaIOS implements FabricaUI {
8     public Boton crearBoton() { return new BotonIOS(); }
9     public TarjetaPropiedad crearTarjetaPropiedad() { return new
    ↪ TarjetaIOS(); }
```

```
10     public FormularioReserva crearFormularioReserva() { return new
        ↪ FormularioIOS(); }
11 }
12
13 class FabricaAndroid implements FabricaUI {
14     public Boton crearBoton() { return new BotonAndroid(); }
15     public TarjetaPropiedad crearTarjetaPropiedad() { return new
        ↪ TarjetaAndroid(); }
16     public FormularioReserva crearFormularioReserva() { return new
        ↪ FormularioAndroid(); }
17 }
```

Client

```
1 class Cliente {
2     private final Boton boton;
3     private final TarjetaPropiedad tarjeta;
4     private final FormularioReserva formulario;
5
6     private final FabricaUI fabrica;
7
8     Cliente(FabricaUI fabrica) {
9         this.boton = fabrica.crearBoton();
10        this.tarjeta = fabrica.crearTarjetaPropiedad();
11        this.formulario = fabrica.crearFormularioReserva();
12        this.fabrica = fabrica;
13    }
14
15    String obtenerVista() {
16        return tarjeta.render() + "\n" + formulario.render() + "\n" +
            ↪ boton.render();
17    }
18
19    @Override
20    public String toString() {
21        if (fabrica instanceof FabricaWeb) {
22            return "WEB";
23        } else if (fabrica instanceof FabricaIOS) {
```

```
24         return "IOS";
25     } else if (fabrica instanceof FabricaAndroid) {
26         return "ANDROID";
27     }
28
29     return "DESCONOCIDO";
30 }
31 }
```

1.7 Conclusiones

El patrón *Abstract Factory* permite crear familias de objetos relacionados manteniendo coherencia entre sus componentes. En este caso, facilita la creación de interfaces específicas para Web, iOS y Android sin mezclar la lógica de negocio con detalles de implementación visual.

Además, reduce el acoplamiento entre el cliente y las clases concretas, permitiendo agregar nuevas plataformas de manera más sencilla. Sin embargo, puede incrementar la cantidad de clases del sistema y hacer más compleja la estructura inicial del proyecto.

Capítulo 2

Builder

2.1 Situación Problema

El motor de búsqueda de Airbnb permite al usuario aplicar una gran cantidad de filtros antes de explorar propiedades: destino, rango de fechas, número de huéspedes, precio mínimo y máximo, tipo de alojamiento, amenidades requeridas, política de cancelación y puntuación mínima, entre otros. No todos los filtros son obligatorios; de hecho, la mayoría de las búsquedas utilizan solo un subconjunto de ellos.

Intentar representar todas las combinaciones posibles mediante constructores sobrecargados resulta inviable: con diez parámetros opcionales se producen decenas de firmas distintas, muchas de ellas ambiguas cuando varios parámetros son del mismo tipo. Pasar `null` para los campos no utilizados es una práctica propensa a errores y difícil de leer. Además, la consulta resultante debe poder validarse como un todo coherente antes de ejecutarse.

2.2 Solución

El patrón Builder separa la construcción del objeto `ConsultaBusqueda` de su representación final. Un objeto `BusquedaBuilder` expone métodos encadenables —`conDestino()`, `conFechas()`, `conPrecioMaximo()`, `conAmenidades()`— que van acumulando los criterios seleccionados por el usuario. Al final, el método `construir()` valida que los criterios sean coherentes y devuelve un objeto `ConsultaBusqueda` completamente configurado.

Esto elimina los constructores con largas listas de parámetros nulos, hace el código de construcción legible y autodescriptivo, y centraliza las reglas de validación en

un único lugar. Si en el futuro se añaden nuevos filtros, solo es necesario agregar el método correspondiente al builder sin afectar al resto del sistema.

Figura 2.1. Diagrama de clases del patrón Abstract Factory

2.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

2.4 Main

Listing 2.1. NombreClase

2.5 Salida del Main

Listing 2.2. NombreClase

2.6 Clases

2.6.1 NombreClase

Listing 2.3. NombreClase

2.6.2 NombreClase

Listing 2.4. NombreClase

2.6.3 NombreClase

Listing 2.5. NombreClase

2.7 Conclusiones

Capítulo 3

Factory Method

3.1 Situación Problema

Airbnb necesita enviar notificaciones a sus usuarios ante distintos eventos: una reserva confirmada, un mensaje de un anfitrión, un recordatorio de check-in o una alerta de pago pendiente. Dependiendo de las preferencias del usuario y del tipo de evento, la notificación puede enviarse por correo electrónico, mensaje push en la aplicación móvil o SMS.

El módulo de eventos actualmente contiene bloques `if-else` que deciden qué objeto de notificación instanciar. Cada vez que se incorpora un nuevo canal —por ejemplo, notificaciones en WhatsApp— es necesario modificar ese módulo central, rompiéndose el principio de abierto/cerrado y aumentando el riesgo de introducir errores en la lógica ya probada. Además, el módulo de eventos no debería tener que conocer los detalles de construcción de cada tipo de notificación.

3.2 Solución

El patrón Factory Method delega la responsabilidad de crear el objeto de notificación a subclases especializadas. Se define una clase base `NotificadorEvento` con un método fábrica abstracto que devuelve un objeto de tipo `Notificacion`; cada subclase concreta (`NotificadorEmail`, `NotificadorPush`, `NotificadorSMS`) implementa ese método y determina qué objeto instanciar.

El módulo de eventos trabaja exclusivamente con la interfaz abstracta y nunca sabe qué tipo concreto está usando. Cuando se requiere un nuevo canal de comunicación, basta con añadir una nueva subclase sin tocar el código existente. Esto hace que el

sistema sea extensible, más fácil de probar y libre de la proliferación de condicionales en el núcleo del negocio.

Figura 3.1. Diagrama de clases del patrón Abstract Factory

3.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

3.4 Main

Listing 3.1. NombreClase

3.5 Salida del Main

Listing 3.2. NombreClase

3.6 Clases

3.6.1 NombreClase

Listing 3.3. NombreClase

3.6.2 NombreClase

Listing 3.4. NombreClase

3.6.3 NombreClase

Listing 3.5. NombreClase

3.7 Conclusiones

Capítulo 4

Prototype

4.1 Situación Problema

Muchos anfitriones de Airbnb administran más de una propiedad y con frecuencia desean publicar alojamientos muy similares entre sí: dos apartamentos en el mismo edificio, o varias habitaciones privadas con las mismas reglas de la casa, los mismos precios base y las mismas amenidades. Actualmente, para registrar una segunda propiedad similar, el anfitrión debe rellenar manualmente todos los campos del formulario desde cero, repitiendo información que ya existe en la primera.

Copiar los datos de una propiedad a mano es tedioso, propenso a inconsistencias y frustrante para el anfitrión. Una solución trivial como compartir la misma referencia de objeto entre dos propiedades es peligrosa: modificar una afectaría a la otra. Se necesita una copia independiente que pueda personalizarse sin riesgo.

4.2 Solución

El patrón Prototype dota a la clase **Propiedad** de un método `clonar()` que produce una copia profunda e independiente del objeto. El anfitrión selecciona una propiedad existente como plantilla y solicita duplicarla; el sistema crea un nuevo objeto con todos los atributos copiados —incluyendo las listas de amenidades y las reglas de la casa— sobre el cual el anfitrión solo necesita ajustar los campos que difieren, como el número de habitación o el título del anuncio.

Esto agiliza considerablemente la publicación de propiedades similares, garantiza que la copia sea totalmente independiente del original y evita duplicar la lógica de

inicialización. El anfitrión obtiene una base sólida sobre la que trabajar en lugar de un formulario vacío.

Figura 4.1. Diagrama de clases del patrón Abstract Factory

4.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

4.4 Main

Listing 4.1. NombreClase

4.5 Salida del Main

Listing 4.2. NombreClase

4.6 Clases

4.6.1 NombreClase

Listing 4.3. NombreClase

4.6.2 NombreClase

Listing 4.4. NombreClase

4.6.3 NombreClase

Listing 4.5. NombreClase

4.7 Conclusiones

Capítulo 5

Singleton

5.1 Situación Problema

La plataforma de Airbnb maneja una serie de parámetros de configuración global que deben ser coherentes en toda la aplicación: la comisión de servicio vigente, las monedas aceptadas, los umbrales de penalización por cancelación y las políticas de reembolso activas. Estos valores los gestiona el equipo de operaciones y pueden cambiar con el tiempo, pero en todo momento debe existir *una única versión de la verdad*.

El problema surge cuando múltiples módulos —el motor de reservas, el servicio de pagos y el panel de anfitriones— instancian de forma independiente su propia copia del objeto de configuración. Si en algún momento se actualiza la comisión de servicio, solo alguna de esas copias recibe el nuevo valor; las demás trabajan con datos desactualizados, produciendo cobros inconsistentes y discrepancias visibles al usuario. Una solución basada en variables globales ordinarias tampoco es suficiente, pues no garantiza que la inicialización ocurra una única vez ni ofrece ningún mecanismo de control de acceso concurrente.

5.2 Solución

El patrón Singleton garantiza que la clase `ConfiguracionGlobal` tenga exactamente una instancia durante todo el ciclo de vida de la aplicación, y proporciona un punto de acceso único a ella. La primera vez que cualquier módulo solicita la configuración, la instancia se crea y se almacena internamente; las solicitudes posteriores reciben siempre la misma referencia.

De esta manera, cuando el equipo de operaciones actualiza la comisión de servicio, el cambio queda reflejado de inmediato para todos los módulos sin necesidad de sincronización adicional. El motor de reservas, el servicio de pagos y el panel de anfitriones consultan el mismo objeto, eliminando las inconsistencias. El patrón también facilita aplicar un mecanismo de control de acceso concurrente en entornos multihilo, ya que la lógica de creación está centralizada en un único lugar.

Figura 5.1. Diagrama de clases del patrón Abstract Factory

5.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

5.4 Main

Listing 5.1. NombreClase

5.5 Salida del Main

Listing 5.2. NombreClase

5.6 Clases

5.6.1 NombreClase

Listing 5.3. NombreClase

5.6.2 NombreClase

Listing 5.4. NombreClase

5.6.3 NombreClase

Listing 5.5. NombreClase

5.7 Conclusiones

Parte II

Patrones Estructurales

Capítulo 6

Adapter

6.1 Situación Problema

Airbnb integra pasarelas de pago de terceros —Stripe, PayPal y Mercado Pago— para procesar los cobros a los huéspedes. Cada pasarela expone su propia interfaz de programación: Stripe utiliza un método `charge()`, PayPal emplea `executePayment()` y Mercado Pago trabaja con `createPaymentIntent()`, todos con firmas, parámetros y formatos de respuesta distintos.

El servicio de pagos interno de Airbnb no puede —ni debería— adaptarse a la interfaz de cada proveedor. Si en el futuro se incorpora una nueva pasarela, habría que modificar el servicio central para entender su vocabulario, acoplando el núcleo del negocio a detalles de implementación externos y fragilizando el sistema.

6.2 Solución

El patrón Adapter introduce una interfaz unificada `PasarelaPago` con un único método `procesarPago()`. Para cada proveedor externo se crea un adaptador concreto —`AdaptadorStripe`, `AdaptadorPayPal`, `AdaptadorMercadoPago`— que implementa esa interfaz y traduce internamente la llamada al método nativo de cada SDK.

El servicio de pagos de Airbnb trabaja exclusivamente contra `PasarelaPago` y desconoce por completo las particularidades de cada proveedor. Añadir una nueva pasarela equivale únicamente a crear un nuevo adaptador, sin tocar el código de negocio. Esto también facilita las pruebas unitarias, pues basta con sustituir el adaptador real por uno simulado.

Figura 6.1. Diagrama de clases del patrón Abstract Factory

6.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

6.4 Main

Listing 6.1. NombreClase

6.5 Salida del Main

Listing 6.2. NombreClase

6.6 Clases

6.6.1 NombreClase

Listing 6.3. NombreClase

6.6.2 NombreClase

Listing 6.4. NombreClase

6.6.3 NombreClase

Listing 6.5. NombreClase

6.7 Conclusiones

Capítulo 7

Bridge

7.1 Situación Problema

El panel de anfitriones de Airbnb permite exportar informes sobre el rendimiento de sus propiedades. Existen distintos *tipos de informe* que un anfitrión puede solicitar: resumen de ingresos, estadísticas de ocupación e historial de reseñas. A su vez, cada informe puede descargarse en distintos *formatos de salida*: PDF para archivar, CSV para importar en hojas de cálculo y JSON para integraciones con herramientas externas de gestión.

Si se modela cada combinación como una clase independiente —`InformeIngresosPDF`, `InformeOcupacionCSV`, `InformeResenasJSON`, etc.— la jerarquía crece de forma combinatoria: tres tipos de informe por tres formatos producen nueve clases, y agregar un nuevo formato obliga a crear una nueva subclase por cada tipo de informe existente. La lógica de generación de datos queda duplicada entre clases que en esencia comparten el mismo proceso de recopilación, diferenciándose solo en cómo serializan el resultado.

7.2 Solución

El patrón Bridge separa la jerarquía de *tipos de informe* —la abstracción— de la jerarquía de *formatos de exportación* —la implementación— conectándolas mediante composición. La clase `InformeAnfitrión` contiene una referencia a un objeto `ExportadorFormato`; los tipos concretos de informe (`InformeIngresos`, `InformeOcupacion`, `InformeResenas`) extienden la abstracción y se encargan de recopilar los datos propios de cada análisis, mientras que los exportadores concretos (`ExportadorPDF`,

`ExportadorCSV`, `ExportadorJSON`) implementan exclusivamente la serialización del resultado en su formato correspondiente.

Ambas dimensiones evolucionan de forma completamente independiente: añadir un nuevo tipo de informe no requiere replicar la lógica de exportación, e incorporar un nuevo formato no impone cambios en ningún informe existente. El número total de clases crece linealmente en lugar de de forma combinatoria, y cada clase mantiene una única responsabilidad bien definida.

Figura 7.1. Diagrama de clases del patrón Abstract Factory

7.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

7.4 Main

Listing 7.1. `NombreClase`

7.5 Salida del Main

Listing 7.2. `NombreClase`

7.6 Clases

7.6.1 `NombreClase`

Listing 7.3. `NombreClase`

7.6.2 NombreClase

Listing 7.4. NombreClase

7.6.3 NombreClase

Listing 7.5. NombreClase

7.7 Conclusiones

Capítulo 8

Composite

8.1 Situación Problema

Algunos anfitriones en Airbnb no gestionan una sola propiedad, sino complejos de alojamiento: un edificio que contiene varios apartamentos, cada uno con sus propias habitaciones y espacios comunes. La plataforma necesita calcular la capacidad total, el precio agregado y el listado de amenidades del conjunto completo, pero también de cada unidad por separado.

Con un modelo plano, el código que consulta la capacidad de «una propiedad» debe saber si está ante una unidad individual o un complejo, y tratar cada caso de forma distinta. Esta distinción se repite en cada operación —cálculo de precio, listado de amenidades, verificación de disponibilidad— generando condicionales duplicados y haciendo el código frágil ante estructuras anidadas de mayor profundidad.

8.2 Solución

El patrón Composite trata los objetos individuales y los compuestos a través de la misma interfaz `ComponenteAlojamiento`, que declara operaciones como `getCapacidad()`, `getPrecio()` y `getAmenidades()`. Las unidades hoja (`Habitacion`, `Apartamento`) implementan estas operaciones directamente; el objeto compuesto (`ComplejoAlojamiento`) las delega recursivamente a sus hijos y agrega los resultados.

El código cliente invoca las mismas operaciones independientemente de si trata con una habitación, un apartamento o un complejo entero. La estructura jerárquica puede anidarse con cualquier profundidad sin que el código de consulta necesite cambiar,

lo que refleja con naturalidad la realidad de los portafolios de propiedades de los anfitriones.

Figura 8.1. Diagrama de clases del patrón Abstract Factory

8.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

8.4 Main

Listing 8.1. NombreClase

8.5 Salida del Main

Listing 8.2. NombreClase

8.6 Clases

8.6.1 NombreClase

Listing 8.3. NombreClase

8.6.2 NombreClase

Listing 8.4. NombreClase

8.6.3 NombreClase

Listing 8.5. NombreClase

8.7 Conclusiones

Capítulo 9

Decorator

9.1 Situación Problema

Al confirmar una reserva en Airbnb, los huéspedes pueden contratar servicios adicionales opcionales: seguro de viaje, traslado al aeropuerto, kit de bienvenida o servicio de limpieza intermedio. Cada combinación de servicios afecta al precio final y a la descripción del paquete reservado.

Modelar todas las combinaciones posibles mediante subclases llevaría a una explosión combinatoria: `ReservaConSeguro`, `ReservaConSeguroYTraslado`, `ReservaConTodosLosServicios` etc. Con solo cuatro servicios opcionales existen dieciséis combinaciones posibles. Incorporar un quinto duplicaría ese número y requeriría modificar la jerarquía de clases cada vez que se añada un servicio nuevo.

9.2 Solución

El patrón Decorator envuelve el objeto `Reserva` base con decoradores concretos: `DecoradorSeguro`, `DecoradorTraslado`, `DecoradorKitBienvenida`. Cada decorador implementa la misma interfaz que `Reserva`, delega las operaciones al objeto que envuelve y añade su propio comportamiento —incrementar el precio, ampliar la descripción— sin modificar la reserva base ni los demás decoradores.

Los servicios se pueden combinar libremente en tiempo de ejecución apilando decoradores en cualquier orden. El código que procesa la reserva no necesita saber cuántos ni cuáles servicios adicionales se han contratado; simplemente invoca las operaciones sobre la interfaz común. Añadir un nuevo servicio es tan sencillo como crear un nuevo decorador.

Figura 9.1. Diagrama de clases del patrón Abstract Factory

9.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

9.4 Main

Listing 9.1. NombreClase

9.5 Salida del Main

Listing 9.2. NombreClase

9.6 Clases

9.6.1 NombreClase

Listing 9.3. NombreClase

9.6.2 NombreClase

Listing 9.4. NombreClase

9.6.3 NombreClase

Listing 9.5. NombreClase

9.7 Conclusiones

Capítulo 10

Facade

10.1 Situación Problema

El proceso de confirmar una reserva en Airbnb involucra internamente varios subsistemas independientes: el *motor de disponibilidad* verifica que las fechas estén libres, el *servicio de pagos* autoriza y retiene el importe, el *servicio de notificaciones* avisa al anfitrión, el *generador de contratos* produce el acuerdo de estancia y el *sistema de calendario* bloquea las fechas.

Cada subsistema expone su propia API con distintas convenciones. El código del cliente —la pantalla de confirmación de la aplicación móvil— debe conocer y orquestar todos estos sistemas en el orden correcto, lo que genera un acoplamiento elevado y hace que cualquier cambio en un subsistema repercuta directamente en la capa de presentación.

10.2 Solución

El patrón Facade introduce la clase **ServicioReserva**, que ofrece un único método **confirmarReserva()** de alto nivel. Internamente, esta fachada coordina la secuencia correcta de llamadas a los subsistemas: verifica disponibilidad, autoriza el pago, genera el contrato, bloquea las fechas y dispara las notificaciones.

La capa de presentación solo conoce **ServicioReserva** y queda desacoplada de los detalles internos. Si el equipo de backend modifica el orden de los pasos o incorpora un nuevo subsistema —como un servicio de detección de fraude— solo es necesario actualizar la fachada, sin tocar la interfaz que consume el cliente.

Figura 10.1. Diagrama de clases del patrón Abstract Factory

10.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

10.4 Main

Listing 10.1. NombreClase

10.5 Salida del Main

Listing 10.2. NombreClase

10.6 Clases

10.6.1 NombreClase

Listing 10.3. NombreClase

10.6.2 NombreClase

Listing 10.4. NombreClase

10.6.3 NombreClase

Listing 10.5. NombreClase

10.7 Conclusiones

Capítulo 11

Flyweight

11.1 Situación Problema

La plataforma de Airbnb almacena en memoria cientos de miles de objetos **Propiedad** para servir búsquedas en tiempo real. Cada propiedad puede declarar hasta cincuenta amenidades: Wi-Fi, piscina, cocina equipada, aire acondicionado, estacionamiento, etc. Si cada objeto **Propiedad** almacenara su propia instancia de cada amenidad —con su nombre, icono, descripción y categoría— la memoria consumida se multiplicaría de forma innecesaria, dado que la inmensa mayoría de propiedades comparte exactamente las mismas amenidades con los mismos datos descriptivos.

El problema se hace crítico cuando el motor de búsqueda carga en caché los resultados para miles de usuarios concurrentes: el consumo de memoria se dispara con objetos masivamente duplicados que son, en esencia, idénticos.

11.2 Solución

El patrón Flyweight separa el estado *intrínseco* de una amenidad —nombre, descripción, icono, categoría, que son inmutables y compartibles— del estado *extrínseco* —si una propiedad concreta la ofrece o no—. Una **FabricaAmenidades** mantiene un mapa de objetos **Amenidad** ya creados; cuando una propiedad declara que ofrece Wi-Fi, recibe la misma instancia compartida que ya usan otras miles de propiedades.

El resultado es una reducción drástica en el número de objetos en memoria: en lugar de crear millones de instancias de **Amenidad** duplicadas, el sistema reutiliza un conjunto acotado de objetos compartidos. Cada propiedad solo guarda referencias a las amenidades relevantes, no copias de sus datos.

Figura 11.1. Diagrama de clases del patrón Abstract Factory

11.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

11.4 Main

Listing 11.1. NombreClase

11.5 Salida del Main

Listing 11.2. NombreClase

11.6 Clases

11.6.1 NombreClase

Listing 11.3. NombreClase

11.6.2 NombreClase

Listing 11.4. NombreClase

11.6.3 NombreClase

Listing 11.5. NombreClase

11.7 Conclusiones

Capítulo 12

Proxy

12.1 Situación Problema

Airbnb almacena información sensible de los anfitriones: número de teléfono personal, dirección exacta del inmueble y datos de contacto directo. Esta información solo debe ser accesible para el huésped una vez que su reserva ha sido *confirmada y pagada*; hasta ese momento, la ficha de la propiedad muestra únicamente datos aproximados —el barrio y la ciudad, pero nunca la dirección exacta ni el teléfono real—.

El problema no es solo de permisos: el módulo que renderiza la ficha de una propiedad no debería tener que conocer las reglas de negocio que determinan qué información mostrar según el estado de la reserva. Si esa lógica recae en la capa de presentación, se repite en cada punto donde se muestran datos del anfitrión, es fácil de omitir y mezcla responsabilidades de visualización con decisiones de acceso.

12.2 Solución

El patrón Proxy introduce `ProxyPerfilAnfitrión`, que implementa exactamente la misma interfaz `PerfilAnfitrión` que el objeto real. El módulo de presentación solo conoce esa interfaz y solicita los datos sin ninguna condición adicional. Internamente, el proxy intercepta cada solicitud, consulta el estado de la reserva y decide de forma autónoma qué devolver: si la reserva está confirmada, delega al objeto real y retorna los datos completos; en caso contrario, retorna una versión censurada donde el teléfono aparece enmascarado y la dirección se reduce al nombre del barrio.

El cliente recibe siempre una respuesta a través de la misma interfaz y en ningún momento toma ni conoce la decisión de qué versión de los datos está viendo. Toda la

política de acceso reside exclusivamente en el proxy, lo que garantiza que no pueda ser ignorada por descuido, centraliza su mantenimiento en un único lugar y deja al servicio real completamente libre de lógica de autorización.

Figura 12.1. Diagrama de clases del patrón Abstract Factory

12.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

12.4 Main

Listing 12.1. NombreClase

12.5 Salida del Main

Listing 12.2. NombreClase

12.6 Clases

12.6.1 NombreClase

Listing 12.3. NombreClase

12.6.2 NombreClase

Listing 12.4. NombreClase

12.6.3 NombreClase

Listing 12.5. NombreClase

12.7 Conclusiones

Parte III

Patrones de Comportamiento

Capítulo 13

Chain Of Responsibility

13.1 Situación Problema

Cuando un anfitrión publica un nuevo alojamiento en Airbnb, la plataforma debe verificar que el anuncio cumple con una serie de requisitos antes de hacerlo visible al público. Estos requisitos son heterogéneos: algunos son puramente técnicos —las fotografías deben superar un umbral mínimo de resolución y cantidad—, otros son de contenido —la descripción no puede contener información de contacto directo que eluda la intermediación de la plataforma—, otros son de coherencia de negocio —el precio base no puede ser anómalamente inferior al promedio de la zona, lo que podría indicar un error del anfitrión— y otros son de cumplimiento legal —ciertas ciudades exigen un número de licencia turística válido antes de permitir la publicación—.

Ejecutar todas estas verificaciones en un único componente mezcla responsabilidades de naturaleza completamente distinta: lógica de procesamiento de imágenes, análisis de texto, comparación de precios de mercado y consulta de registros regulatorios conviven en el mismo lugar. Cualquier cambio en la normativa de una ciudad concreta, o la incorporación de un nuevo criterio de calidad, obliga a intervenir ese componente central con el riesgo de afectar verificaciones que no tenían relación con el cambio.

13.2 Solución

El patrón Chain of Responsibility organiza cada verificación como un manejador independiente: `VerificadorFotos`, `VerificadorContenidoDescripcion`, `VerificadorCoherenciaPr` y `VerificadorLicenciaTuristica`. Cada manejador tiene una responsabilidad acotada: evalúa el aspecto del anuncio que le corresponde y, si detecta un incumplimiento,

detiene la cadena y devuelve al anfitrión un motivo de rechazo claro y específico. Si el anuncio supera su verificación, lo pasa al siguiente manejador sin intervención adicional.

Solo cuando el anuncio recorre la cadena completa sin ser detenido se procede a su publicación. Cada verificador es completamente ajeno a los criterios de los demás, por lo que modificar el umbral de calidad fotográfica no tiene ningún efecto sobre la verificación de licencias. Incorporar una nueva exigencia —por ejemplo, un detector de precios discriminatorios— se reduce a insertar un nuevo manejador en la cadena en el punto que corresponda, sin tocar ninguno de los existentes.

13.3 Diagrama de clases

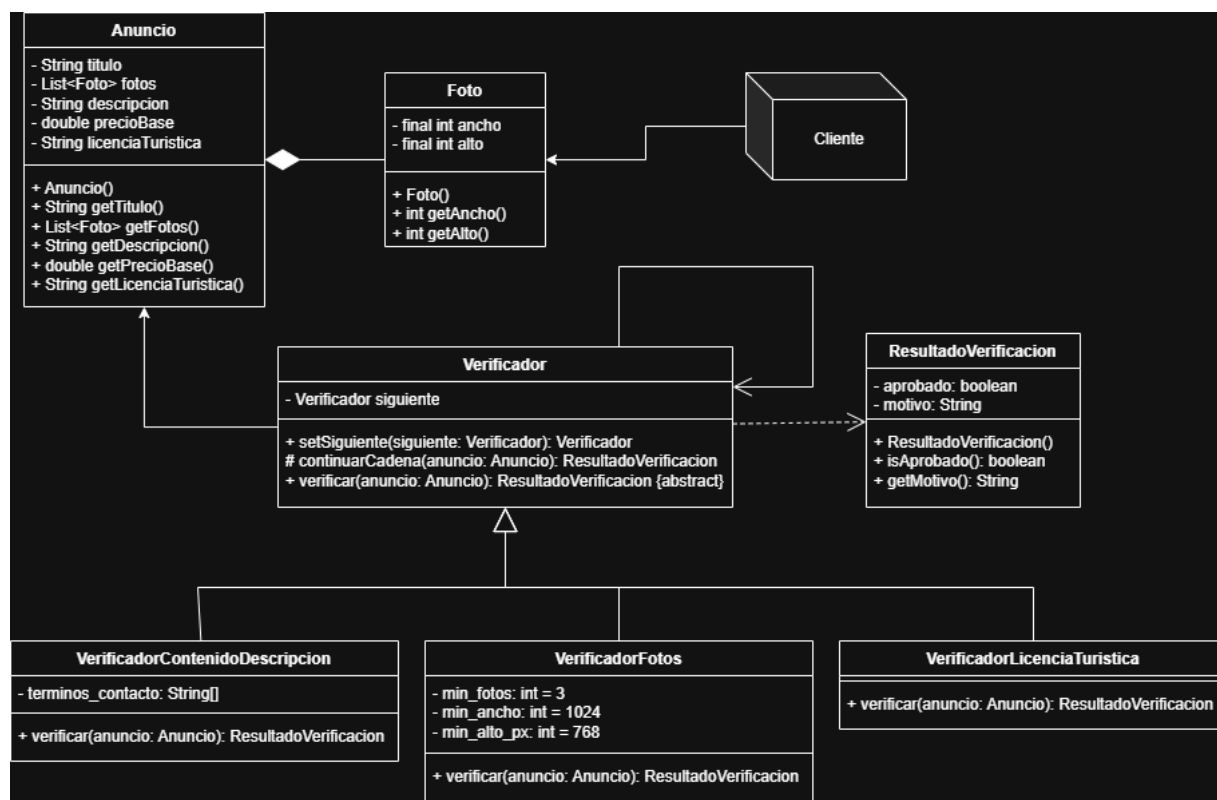


Figura 13.1. Diagrama de clases del patrón Chain of Responsibility

13.4 Roles de las Clases

- **Handler:**
- **ConcreteHandler:**

- Client:

13.5 Main

ChainOfResponsability

```
1 // ChainOfResponsability
2 package behavioral.chainofresponsability;
3
4 import java.util.List;
5
6 class ChainOfResponsability {
7     public static void main(String[] args) {
8         Cliente cliente = new Cliente();
9
10        Anuncio[] anuncios = {
11            new Anuncio("Apartamento en el centro historico",
12                List.of(new Foto(1920, 1080), new Foto(1280,
13                    ↪ 960), new Foto(1600, 900)),
14                "Precioso apartamento con todas las
15                    ↪ comodidades. Ideal para parejas.",
16                80.0, "LIC-2024-BCN-00412"),
17            new Anuncio("Estudio acogedor",
18                List.of(new Foto(1920, 1080)),
19                "Estudio luminoso en zona tranquila.",
20                60.0, "LIC-2024-BCN-00999"),
21            new Anuncio("Piso moderno",
22                List.of(new Foto(1920, 1080), new Foto(1280,
23                    ↪ 960), new Foto(1600, 900)),
24                "Piso muy bonito. Contactame por whatsapp
25                    ↪ para mas info.",
26                90.0, "LIC-2024-BCN-00300"),
27            new Anuncio("Villa con piscina",
28                List.of(new Foto(1920, 1080), new Foto(1280,
29                    ↪ 960), new Foto(1600, 900)),
30                "Impresionante villa con jardin y piscina
31                    ↪ privada.",
32                200.0, "")
33        };
34    }
35}
```

```

27         };
28
29         System.out.println("VERIFICACION DE ANUNCIOS AIRBNB\n");
30
31         for (Anuncio anuncio : anuncios) {
32             ResultadoVerificacion resultado =
33                 ↪ cliente.procesar(anuncio);
34             mostrarResultado(anuncio, resultado);
35         }
36
37     private static void mostrarResultado(Anuncio anuncio,
38         ↪ ResultadoVerificacion resultado) {
39         System.out.println("Anuncio : " + anuncio.getTitulo());
40         System.out.println("Estado : " + (resultado.isAprobado() ?
41             ↪ "APROBADO" : "RECHAZADO"));
42         System.out.println("Motivo : " + resultado.getMotivo());
43         System.out.println("-".repeat(55));
44     }
45 }

```

13.6 Salida del Main

```

VERIFICACION DE ANUNCIOS AIRBNB

Anuncio : Apartamento en el centro historico
Estado   : APROBADO
Motivo   : Anuncio aprobado para publicacion.
-----

Anuncio : Estudio acogedor
Estado   : RECHAZADO
Motivo   : Se requieren al menos 3 fotografias.
-----

Anuncio : Piso moderno
Estado   : RECHAZADO
Motivo   : La descripcion contiene informacion de contacto no
          permitida.

```

```
-----  
Anuncio : Villa con piscina  
Estado  : RECHAZADO  
Motivo   : Se requiere un numero de licencia turistica valido  
          para publicar en esta ciudad.  
-----
```

13.7 Clases

13.7.1 NombreClase

Main.java

1

13.7.2 NombreClase

Main.java

1

13.7.3 NombreClase

Main.java

1

13.8 Conclusiones

Capítulo 14

Command

14.1 Situación Problema

14.2 Solución

Figura 14.1. Diagrama de clases del patrón Abstract Factory

14.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

14.4 Main

Listing 14.1. NombreClase

14.5 Salida del Main

Listing 14.2. NombreClase

14.6 Clases

14.6.1 NombreClase

Listing 14.3. NombreClase

14.6.2 NombreClase

Listing 14.4. NombreClase

14.6.3 NombreClase

Listing 14.5. NombreClase

14.7 Conclusiones

Capítulo 15

Iterator

15.1 Situación Problema

Airbnb mantiene las propiedades disponibles en distintas estructuras de datos internas según el contexto: una lista ordenada por relevancia para los resultados de búsqueda, un árbol geoespacial para la vista de mapa y un grafo de recomendaciones para la sección «también te podría gustar». Los distintos módulos de la plataforma —exportación de datos, motor de recomendaciones, generador de reportes— necesitan recorrer estas colecciones de propiedades de forma uniforme, sin acoplarse a su estructura interna.

Si cada módulo accede directamente a la representación interna de la colección, cualquier cambio en la estructura de datos obligaría a actualizar todos los módulos que la recorren, creando un acoplamiento innecesario y frágil.

15.2 Solución

El patrón Iterator define la interfaz `IteradorPropiedades` con los métodos `siguiente()` y `hayMas()`. Cada estructura de datos proporciona su propio iterador concreto que sabe cómo recorrerla sin exponer su representación interna. Los módulos clientes utilizan únicamente la interfaz del iterador y son agnósticos a si están traversando una lista, un árbol o un grafo.

Cuando el equipo de backend decide cambiar la estructura interna de almacenamiento de propiedades, basta con actualizar el iterador correspondiente. El código de los módulos consumidores no sufre ningún cambio, manteniéndose el principio de encapsulamiento de las colecciones.

Figura 15.1. Diagrama de clases del patrón Abstract Factory

15.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

15.4 Main

Listing 15.1. NombreClase

15.5 Salida del Main

Listing 15.2. NombreClase

15.6 Clases

15.6.1 NombreClase

Listing 15.3. NombreClase

15.6.2 NombreClase

Listing 15.4. NombreClase

15.6.3 NombreClase

Listing 15.5. NombreClase

15.7 Conclusiones

Capítulo 16

Mediator

16.1 Situación Problema

El sistema de mensajería de Airbnb conecta a huéspedes y anfitriones dentro de la plataforma. Sin embargo, los mensajes no son simples textos: pueden activar acciones automáticas. Cuando el anfitrión responde una consulta de disponibilidad, el sistema debe verificar el calendario; cuando un huésped confirma verbalmente el check-in, el sistema puede actualizar el estado de la reserva; si alguna de las partes reporta un problema, se abre automáticamente un ticket de soporte.

Si cada participante del chat —huésped, anfitrión, sistema de reservas, soporte— se comunica directamente con los demás, se crea una red densa de dependencias cruzadas. Añadir un nuevo tipo de participante o una nueva acción automática requiere modificar todas las partes que podrían interactuar con él.

16.2 Solución

El patrón Mediator introduce **CentralMensajería** como el único punto de coordinación. Huéspedes, anfitriones y servicios automáticos solo se comunican con el mediador, nunca directamente entre sí. El mediador conoce las reglas de negocio: qué debe ocurrir cuando se envía cierto tipo de mensaje y qué participantes deben ser notificados o activados en consecuencia.

Esto reduce las dependencias entre participantes de $O(n^2)$ a $O(n)$, hace explícitas y centralizadas las reglas de interacción, y permite incorporar nuevos participantes o nuevas reglas de automatización modificando únicamente el mediador.

Figura 16.1. Diagrama de clases del patrón Abstract Factory

16.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

16.4 Main

Listing 16.1. NombreClase

16.5 Salida del Main

Listing 16.2. NombreClase

16.6 Clases

16.6.1 NombreClase

Listing 16.3. NombreClase

16.6.2 NombreClase

Listing 16.4. NombreClase

16.6.3 NombreClase

Listing 16.5. NombreClase

16.7 Conclusiones

Capítulo 17

Memento

17.1 Situación Problema

Cuando un usuario de Airbnb configura una búsqueda con múltiples filtros —destino, fechas, tipo de alojamiento, amenidades— y luego navega hacia la ficha de una propiedad, al presionar el botón «atrás» espera que su búsqueda esté exactamente como la dejó: los mismos filtros activos, el mismo punto de desplazamiento en la lista de resultados y la misma vista seleccionada (mapa o lista). Esto mismo aplica cuando el usuario interrumpe una reserva a mitad del proceso multistep y regresa más tarde.

Restablecer el estado de la búsqueda requiere conservar todos sus atributos en un momento específico. Exponer el estado interno del objeto `BusquedaActiva` para que capas externas lo almacenen viola el encapsulamiento y acopla el mecanismo de almacenamiento a los detalles internos del objeto.

17.2 Solución

El patrón Memento permite que `BusquedaActiva` produzca un objeto `InstantaneaBusqueda` que captura su estado interno completo sin exponerlo al exterior. El `GestorNavegacion` almacena estas instantáneas en una pila; cuando el usuario regresa, solicita la instantánea correspondiente y `BusquedaActiva` se restaura al estado guardado.

El encapsulamiento del objeto de búsqueda se mantiene intacto —el gestor maneja instantáneas opacas—, y la experiencia del usuario es fluida al retomar exactamente donde lo dejó, independientemente de cuántos pasos de navegación hayan ocurrido en el intermedio.

Figura 17.1. Diagrama de clases del patrón Abstract Factory

17.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

17.4 Main

Listing 17.1. NombreClase

17.5 Salida del Main

Listing 17.2. NombreClase

17.6 Clases

17.6.1 NombreClase

Listing 17.3. NombreClase

17.6.2 NombreClase

Listing 17.4. NombreClase

17.6.3 NombreClase

Listing 17.5. NombreClase

17.7 Conclusiones

Capítulo 18

Observer

Tipo:

GoF (pág.):

18.1 Situación Problema

18.2 Solución

Figura 18.1. Diagrama de clases del patrón Abstract Factory

18.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

18.4 Main

Listing 18.1. NombreClase

18.5 Salida del Main

Listing 18.2. NombreClase

18.6 Clases

18.6.1 NombreClase

Listing 18.3. NombreClase

18.6.2 NombreClase

Listing 18.4. NombreClase

18.6.3 NombreClase

Listing 18.5. NombreClase

18.7 Conclusiones

Capítulo 19

State

19.1 Situación Problema

Una reserva en Airbnb atraviesa varios estados a lo largo de su ciclo de vida: *solicitada*, *confirmada*, *en curso*, *completada* y *cancelada*. El comportamiento permitido varía drásticamente según el estado: una reserva solicitada puede ser rechazada por el anfitrión, pero una completada no; se puede cancelar una reserva confirmada, pero con penalizaciones distintas según la antelación; no es posible dejar reseña hasta que la reserva esté completada.

Implementar estas reglas con condicionales sobre un atributo de estado produce un código lleno de `if-else` anidados que crece con cada nuevo estado o transición. El riesgo de omitir una combinación inválida es alto, y el código resulta difícil de leer y extender.

19.2 Solución

El patrón State representa cada estado del ciclo de vida como una clase concreta que implementa la interfaz `EstadoReserva`. La clase `Reserva` delega la respuesta a cada acción —`confirmar()`, `cancelar()`, `completar()`, `dejarResena()`— al objeto de estado que tiene asignado en ese momento. Cada estado concreto sabe exactamente qué transiciones son válidas, qué lógica ejecutar y a qué estado siguiente moverse.

El código de cada estado es compacto y autocontenido. Añadir un nuevo estado o modificar las reglas de transición de uno existente no requiere alterar los demás. La clase `Reserva` queda libre de condicionales y concentrada en su responsabilidad principal.

Figura 19.1. Diagrama de clases del patrón Abstract Factory

19.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

19.4 Main

Listing 19.1. NombreClase

19.5 Salida del Main

Listing 19.2. NombreClase

19.6 Clases

19.6.1 NombreClase

Listing 19.3. NombreClase

19.6.2 NombreClase

Listing 19.4. NombreClase

19.6.3 NombreClase

Listing 19.5. NombreClase

19.7 Conclusiones

Capítulo 20

Strategy

20.1 Situación Problema

Airbnb aplica precios dinámicos a sus propiedades: el precio por noche varía según el algoritmo de tarificación configurado por el anfitrión. Algunos anfitriones optan por *precios estacionales* —sube en verano y festivos—, otros por *precios basados en demanda* —ajuste automático según ocupación en el área— y otros por *descuentos por estancia prolongada* —reducción progresiva para reservas de más de siete noches—.

Si toda esta lógica reside en la clase **Propiedad** mediante una cadena de condicionales, agregar una nueva estrategia de precios implica modificar esa clase y el riesgo de que cambios en una estrategia afecten accidentalmente a las otras. Además, los anfitriones pueden querer cambiar su estrategia de precios en cualquier momento sin afectar la disponibilidad de la propiedad.

20.2 Solución

El patrón Strategy extrae cada algoritmo de tarificación a una clase independiente que implementa la interfaz **EstrategiaPrecios** con el método `calcularPrecio()`. La clase **Propiedad** mantiene una referencia a la estrategia activa y la delega para calcular el precio de cualquier noche consultada.

El anfitrión puede cambiar su estrategia en tiempo de ejecución sin más que sustituir el objeto de estrategia. Cada algoritmo queda aislado, es probable de forma independiente y puede incorporarse una nueva estrategia sin modificar **Propiedad** ni las estrategias existentes.

Figura 20.1. Diagrama de clases del patrón Abstract Factory

20.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

20.4 Main

Listing 20.1. NombreClase

20.5 Salida del Main

Listing 20.2. NombreClase

20.6 Clases

20.6.1 NombreClase

Listing 20.3. NombreClase

20.6.2 NombreClase

Listing 20.4. NombreClase

20.6.3 NombreClase

Listing 20.5. NombreClase

20.7 Conclusiones

Capítulo 21

Template Method

21.1 Situación Problema

El proceso de publicación de un nuevo alojamiento en Airbnb sigue siempre los mismos pasos generales: validar los datos básicos, subir y verificar las fotografías, configurar el calendario de disponibilidad, definir el precio base y finalmente publicar el anuncio. Sin embargo, algunos pasos varían según el tipo de alojamiento: publicar una *experiencia* requiere validar las credenciales del anfitrión-guía; publicar una *propiedad de lujo* activa una revisión adicional por parte de un equipo especializado.

Si cada tipo de alojamiento reimplementa el flujo completo de publicación, se duplica la lógica común y aumenta el riesgo de que los tipos difieran en pasos que deberían ser idénticos, como la verificación de fotografías o la publicación final.

21.2 Solución

El patrón Template Method define en la clase base `ProcesoPublicacion` el esqueleto del algoritmo de publicación como un método `publicar()` que llama en orden a los pasos del proceso. Los pasos invariantes —validación de datos, verificación de fotos, publicación final— se implementan directamente en la clase base; los pasos específicos de cada tipo se declaran como métodos abstractos que las subclases concretas (`PublicacionExperiencia`, `PublicacionPropiedadLujo`) implementan a su manera.

La secuencia global está protegida en un único lugar y no puede ser alterada por las subclases, garantizando consistencia. Solo los puntos de variación reales quedan abiertos a especialización, evitando la duplicación del esqueleto del proceso.

Figura 21.1. Diagrama de clases del patrón Abstract Factory

21.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

21.4 Main

Listing 21.1. NombreClase

21.5 Salida del Main

Listing 21.2. NombreClase

21.6 Clases

21.6.1 NombreClase

Listing 21.3. NombreClase

21.6.2 NombreClase

Listing 21.4. NombreClase

21.6.3 NombreClase

Listing 21.5. NombreClase

21.7 Conclusiones

Capítulo 22

Visitor

22.1 Situación Problema

El equipo de analítica de Airbnb necesita ejecutar distintos tipos de análisis sobre el catálogo de alojamientos: calcular el ingreso potencial máximo de cada propiedad, evaluar el índice de cumplimiento de estándares de calidad y generar un reporte fiscal con la información relevante para declaraciones tributarias en cada país. Cada análisis opera sobre los mismos objetos del dominio —`Habitacion`, `Apartamento`, `CasaRural`— pero extrae y procesa datos distintos.

Añadir la lógica de cada análisis directamente a las clases del dominio las contamina con responsabilidades que no le corresponden. Cada vez que el equipo de analítica requiere un nuevo tipo de informe, hay que modificar todas las clases de alojamiento, rompiendo el principio de responsabilidad única y arriesgando errores en el núcleo del modelo de dominio.

22.2 Solución

El patrón Visitor separa los algoritmos de análisis de las clases sobre las que operan. Cada clase de alojamiento implementa un método `aceptar(visitorante)` que simplemente invoca el método correspondiente del visitante. Cada análisis se implementa como un visitante concreto —`VisitanteIngresoMaximo`, `VisitanteCalidad`, `VisitanteFiscal`— con métodos específicos para cada tipo de alojamiento.

Las clases del dominio no necesitan cambiar cuando se requiere un nuevo análisis; basta con crear un nuevo visitante. Cada visitante encapsula de forma cohesiva toda

la lógica de su análisis particular, y la estructura del modelo de dominio permanece limpia y estable.

Figura 22.1. Diagrama de clases del patrón Abstract Factory

22.3 Roles de las Clases

- **AbstractFactory:**
- **ConcreteFactory:**
- **AbstractProduct:**
- **ConcreteProduct:**
- **Client:**

22.4 Main

Listing 22.1. NombreClase

22.5 Salida del Main

Listing 22.2. NombreClase

22.6 Clases

22.6.1 NombreClase

Listing 22.3. NombreClase

22.6.2 NombreClase

Listing 22.4. NombreClase

22.6.3 NombreClase

Listing 22.5. NombreClase

22.7 Conclusiones

Conclusiones Generales

A lo largo de este documento se analizaron los 23 patrones de diseño del catálogo GoF, cada uno aplicado a una problemática concreta dentro de la plataforma Airbnb. A continuación se presentan las reflexiones transversales más relevantes.

Sobre los patrones creacionales

[Síntesis de los 5 patrones creacionales: qué tienen en común, cuándo preferir uno sobre otro dentro de una plataforma como Airbnb.]

Sobre los patrones estructurales

[Síntesis de los 7 patrones estructurales: cómo facilitan la integración de módulos y servicios externos en una arquitectura de microservicios o monolítica.]

Sobre los patrones de comportamiento

[Síntesis de los 10 patrones de comportamiento: cómo mejoran la colaboración entre objetos y la flexibilidad ante cambios de requisitos.]

Reflexión final

[Conclusión global: importancia de conocer y aplicar patrones de diseño como herramienta de comunicación y de arquitectura de software en proyectos reales.]